Informe de Sistema distribuido para servicios de computación de alto rendimiento (HPC) – UPB-CIENTÍFICA

Andrés Felipe Pinzón Camacho, Sergio Alejandro Sanabria Moreno, Angie Lorena Villareal Vásquez, Valentina Téllez Salazar, Fernando Cancelado Figueroa y Daniel Felipe Aguilar Latorre

Universidad Pontificia Bolivariana Seccional Bucaramanga, COLOMBIA

Resumen. Este artículo presenta el diseño y la implementación de un sistema distribuido orientado a dotar a la microempresa UPB-CIENTÍFICA de servicios de computación de alto rendimiento (HPC) y de gestión colaborativa de archivos, soportado en una infraestructura de cómputo en malla (Grid) alojada en el Centro de Computación Avanzada de la Universidad Pontificia Bolivariana seccional Bucaramanga. El sistema recibe código fuente y conjuntos de datos y los ejecuta en paralelo con MPI a través de un gestor del clúster en Java RMI, e integra los servicios complementarios de la Figura 1 del enunciado —almacenamiento distribuido de archivos, fotos y videos; sincronización de archivos mediante gRPC; compartición de archivos con permisos Unix; álbum de fotografías y streaming de video— bajo un esquema común de autenticación centralizada (servicio SOAP sobre un directorio LDAP), seguridad informática y monitoreo. Se presenta el estado del arte que sustenta las decisiones tecnológicas del proyecto, el marco conceptual, los objetivos, la metodología adoptada, los resultados obtenidos y el cronograma del periodo académico 2026-20.

Palabras clave: sistemas distribuidos, computación de alto rendimiento, computación en malla, RMI, gRPC, SOAP, MPI.

1 Introducción

La computación de alto rendimiento (HPC) se ha convertido en un recurso estratégico para la comunidad científica, al permitir el procesamiento de grandes volúmenes de datos y la ejecución de simulaciones complejas en tiempos razonables. Sin embargo, poner esta capacidad de cómputo al servicio de investigadores externos exige mucho más que un clúster de máquinas: requiere una infraestructura distribuida capaz de administrar el acceso, almacenar y sincronizar los datos de trabajo, compartir resultados de forma segura y monitorear continuamente su disponibilidad.

Este artículo presenta el sistema distribuido de la microempresa UPB-CIENTÍFICA, dedicada a ofrecer servicios de HPC a la comunidad científica mediante el procesamiento paralelo de código basado en MPI. Como parte del proyecto de aula de la asignatura Sistemas Distribuidos (periodo 2026-20) de la Universidad Pontificia Bolivariana seccional Bucaramanga, el sistema integra una infraestructura de cómputo en malla (Grid), capaz de sumar al clúster computadoras, dispositivos móviles y elementos de IoT tanto desde el centro de cómputo como de forma externa, con servicios complementarios de almacenamiento distribuido, sincronización, compartición, álbum de fotografías y streaming, todos alojados en el Centro de Computación Avanzada de la universidad y desarrollados exclusivamente con software libre. Los usuarios acceden a ellos desde clientes nativos para Windows, Linux y Android y desde una interfaz web disponible en cualquier navegador.

El documento se organiza de la siguiente manera: la sección 2 presenta el estado del arte que sustenta las decisiones tecnológicas del proyecto; la sección 3 expone el marco conceptual; la sección 4 detalla los objetivos generales y específicos; la sección 5 describe la metodología de desarrollo adoptada; las secciones 6 y 7 presentan los resultados obtenidos y el cronograma; y la sección 8 cierra con la discusión y las conclusiones del trabajo.

2 Estado del arte

A continuación, se presentan los ejes temáticos clave que sustentan el diseño técnico del proyecto, organizados según los componentes tecnológicos exigidos por la situación problemática de UPB-CIENTÍFICA: computación en malla, middleware de objetos distribuidos, protocolos de comunicación entre servicios, sincronización y compartición de archivos, streaming de video, autenticación centralizada, seguridad, monitoreo de infraestructura HPC, cómputo paralelo con MPI y tolerancia a fallos.

2.1 Computación en malla y su comparación con la nube

Adepoju y Ologunagba (2025) presentan una revisión sistemática que compara la evolución de la computación en malla frente a las infraestructuras modernas de computación en la nube, examinando arquitecturas, aplicaciones y tendencias emergentes de ambos paradigmas. Los autores destacan que, si bien la nube ofrece elasticidad y facilidad de aprovisionamiento, la computación en malla conserva ventajas en escenarios donde se requiere integrar recursos heterogéneos y geográficamente distribuidos —como computadores, dispositivos móviles y equipos de laboratorio— bajo un esquema de propiedad compartida entre instituciones, en lugar de depender de un proveedor comercial único. En un sentido similar, Sungkar y Kogoya (2020) describen la computación Grid como un tipo de sistema paralelo y distribuido que permite compartir, seleccionar y agregar recursos autónomos dispersos geográficamente en tiempo de ejecución, según su disponibilidad, capacidad y los requisitos de calidad de servicio del usuario. Para UPB-CIENTÍFICA estos hallazgos son directamente aplicables: la exigencia de integrar computadoras, dispositivos móviles e IoT al clúster HPC, tanto interna como externamente, corresponde al modelo de computación en malla descrito en ambos estudios, y no a un esquema de nube pública, que además está descartado por las restricciones presupuestales del proyecto.

2.2 Middleware de objetos distribuidos: RMI y CORBA

La Figura 1 del enunciado asigna Java RMI al servidor del clúster (“RMI Server Java Cluster”), por lo que el gestor que recibe, planifica y coordina los trabajos MPI exige comprender las fortalezas y limitaciones de RMI frente a alternativas como CORBA. Munoz y Zalewski (2001) realizan una comparación arquitectónica y de desempeño entre RMI y CORBA sobre plataformas Solaris y Windows NT, midiendo el tiempo de respuesta ante invocaciones de métodos remotos con distinto número de parámetros y bajo carga de múltiples clientes concurrentes. Juric (2000) complementa este análisis con una comparación cualitativa y cuantitativa centrada en la facilidad de desarrollo, la madurez y el desempeño de RMI frente a CORBA en aplicaciones basadas en Java, y concluye que RMI resulta más sencillo de integrar en entornos homogéneos Java, mientras que CORBA maneja mejor la interoperabilidad multilenguaje con mayor sobrecarga. Estos hallazgos justifican, dentro de la arquitectura de UPB-CIENTÍFICA, el uso de Java RMI en el gestor del clúster HPC: sus componentes (gestor, fachada de trabajos y ejecutor en el nodo maestro) están escritos en Java y no requieren

interoperabilidad multilenguaje en ese tramo; el resto de clientes accede a él a través de una fachada REST.

2.3 Protocolos de comunicación entre servicios: SOAP, REST y gRPC

El sistema integra los estilos de comunicación que muestra la Figura 1 del enunciado: SOAP (servidor PHP de usuarios y autenticación), un servidor web basado en JavaScript, Java RMI (gestor del clúster) y gRPC (servidor Go de sincronización de archivos), además de APIs REST para los servicios de implementación. Castillo, Bernier, Arenas, Merelo y García-Sánchez (2011) comparan experimentalmente SOAP y REST implementando un algoritmo genético maestro-esclavo sobre ambos protocolos: SOAP resulta conceptualmente más complejo y pesado, pero ofrece un contrato de interfaz más formal (WSDL), adecuado para un servicio de identidad que consumen todos los demás; REST resulta más liviano para escenarios de alta frecuencia de solicitudes. Por su parte, la literatura técnica reciente coincide en que gRPC, al basarse en HTTP/2 y en la serialización binaria de Protocol Buffers, ofrece menor latencia y mayor throughput que SOAP o REST, y es especialmente adecuado para transferencias en flujo (streaming) bidireccional, lo que explica su elección para el servicio de sincronización de archivos, que transfiere árboles de directorios y contenido binario entre los clientes y el servidor.

2.4 Sincronización y compartición de archivos distribuidos

El servicio File Sync debe crear en el servidor una copia síncrona de seguridad de un directorio local del cliente, de forma programada en el escritorio y al alcanzar la red Wi-Fi local en el móvil, manteniendo consistencia entre todas las copias. Mehdi y Ragab-Hassen (2016) presentan una encuesta de tecnologías de sincronización de archivos, tanto centralizadas en la nube (Dropbox, Google Drive) como basadas en P2P, y discuten sus fortalezas y debilidades en términos de ancho de banda, latencia y consistencia; destacan que los protocolos que solo transmiten los cambios reducen significativamente el consumo de red, principio aplicable al servicio de copia síncrona exigido por el proyecto. Complementariamente, el estudio cualitativo de Capra (2014), basado en entrevistas a usuarios universitarios, identifica limitaciones prácticas de la sincronización automática, como la imposibilidad de sincronizar carpetas parcialmente o la necesidad de procesos manuales de respaldo, relevantes para el diseño del Home parametrizado que exige UPB-CIENTÍFICA para cada cliente.

2.5 Arquitecturas de streaming de video adaptativo

El servicio de Streaming debe ofrecer reproducción fluida, control de acceso y gestión de permisos para los videos que los científicos comparten en presentaciones empresariales. Natali y Merani (2015) proponen una arquitectura de streaming adaptativo (P2P-DASH) que combina Dynamic Adaptive Streaming over HTTP con redes peer-to-peer y una estrategia de control de tasa distribuida que ajusta la calidad del video según las condiciones de la red. En una línea similar, Waheed et al. (2021) presentan una arquitectura eficiente para streaming en tasa de bits adaptativa, y demuestran que el consumo de CPU y energía puede optimizarse ajustando el tamaño de segmento sin sacrificar la calidad percibida. Ambos estudios orientan el diseño del servicio de streaming, que debe equilibrar la calidad de reproducción y el consumo de recursos del servidor, dado que la infraestructura se aloja íntegramente en el Centro de Computación Avanzada sin redes de distribución de contenido externas.

2.6 Autenticación centralizada y directorios activos

UPB-CIENTÍFICA exige que los perfiles de usuario —autenticación, autorización, mapa de servicios y árboles de archivos— sean administrados por un servicio de directorio activo. El trabajo clásico de Woo y Lam (1992) sienta las bases teóricas de la autenticación en sistemas distribuidos, formalizando los requisitos de un protocolo seguro frente a ataques de intermediario y repetición, principios vigentes en los protocolos que usan servicios de directorio como LDAP y Active Directory. La literatura técnica reciente coincide en que estas tecnologías siguen siendo fundacionales para la gestión centralizada de identidades. Esto sustenta la decisión de emplear un directorio OpenLDAP, expuesto mediante el servidor SOAP en PHP de la Figura 1, como fuente única de verdad para los perfiles compartidos entre los servicios del sistema (File Sync, Shared File, Photo Album, Streaming y MPI Jobs).

2.7 Seguridad en sistemas distribuidos

El proyecto exige cifrado asimétrico GPG para las copias de seguridad, autenticación de los clientes basada en token o múltiple factor, y un análisis de vulnerabilidades de la aplicación web en su ambiente de producción con los niveles de riesgo asumidos claramente documentados. Priyadarshini (2022) divide el problema de la seguridad distribuida en dos frentes: la seguridad de la comunicación entre usuarios y procesos —autenticación, integridad y cifrado— y el control de los permisos de acceso a los recursos. Babu, Abraham y Borasia (2013) revisan la criptografía de clave simétrica y enumeran los pilares de la seguridad distribuida: autenticación, confidencialidad, autorización, integridad, disponibilidad y trazabilidad. Estos marcos son aplicables al diseño de seguridad de UPB-CIENTÍFICA, en el que conviven el cifrado asimétrico de las copias (confidencialidad de datos en reposo) con tokens de sesión firmados (verificación de identidad) y permisos Unix sobre los archivos (autorización).

2.8 Monitoreo de clústeres e infraestructuras HPC

El módulo de monitoreo debe hacer seguimiento al servicio HPC y a los demás componentes de la infraestructura, con estadísticas en tiempo real de ocupación de recursos (CPU, memoria, almacenamiento), carga de trabajos, tiempos de ejecución y disponibilidad de servicios, accesible vía web, con reportes históricos y alertas configurables. Kunz (2022) implementa una solución de monitoreo de trabajos HPC basada en SLURM, Prometheus y Grafana, y evalúa marcos como PIKA y ClusterCockpit. La documentación técnica de referencia coincide en que este patrón —exportadores de métricas por nodo que alimentan una base de series temporales, con paneles de visualización y alertas configurables— es el estándar de facto para clústeres on-premise, lo que sustenta la elección de Prometheus y Grafana, ambos de software libre.

2.9 MPI y paradigmas de cómputo paralelo

AlHafez y Kurdi (2025) comparan MPI, OpenMP y CUDA y muestran que MPI sobresale en entornos de memoria distribuida, con una escalabilidad casi lineal en aplicaciones intensivas en comunicación entre nodos, mientras que OpenMP resulta más adecuado para el paralelismo de memoria compartida dentro de un nodo. Esto confirma que MPI es la tecnología correcta para coordinar el procesamiento entre los nodos del clúster de la infraestructura Grid, reservando los paradigmas de memoria compartida para la optimización interna de cada nodo.

2.10 Tolerancia a fallos y replicación

Aunque el enunciado no detalla mecanismos de replicación, la carta descriptiva del curso (UPB, 2020) incluye la tolerancia a fallos y la alta disponibilidad como aspectos transversales a cualquier sistema distribuido. El trabajo tutorial de Guerraoui y Schiper (1996) define la linealizabilidad como criterio de corrección para servicios replicados y presenta la replicación primario-respaldo y la replicación activa. Estos conceptos son pertinentes en dos frentes: el repositorio distribuido de archivos, fotos y videos podría beneficiarse de réplicas para tolerar la caída de un nodo, y el clúster HPC, al ejecutar trabajos de larga duración, requiere mecanismos de recuperación ante fallos (reintento de trabajos, reasignación de nodos caídos).

3 Marco conceptual

3.1 Sistema distribuido

Conjunto de nodos autónomos que se comunican por red y se presentan ante el usuario como un único sistema coherente, coordinando su funcionamiento sin depender de un reloj o una memoria compartida.

3.2 Computación en malla (Grid computing)

Modelo de sistema distribuido en el que se comparten, seleccionan y agregan recursos autónomos y geográficamente dispersos —computadoras, dispositivos móviles y elementos de IoT— según su disponibilidad, capacidad y los requisitos de calidad de servicio del usuario, sin depender de un proveedor comercial único.

3.3 Computación de alto rendimiento (HPC)

Subconjunto de la computación distribuida orientado a maximizar la capacidad de procesamiento mediante clústeres de múltiples nodos, empleados para ejecutar trabajos científicos, de ingeniería o de análisis de datos a gran escala.

3.4 MPI

Interfaz de paso de mensajes (Message Passing Interface) que coordina la ejecución de un mismo programa entre múltiples procesos que no comparten memoria.

3.5 Middleware de objetos distribuidos

Capas de software (RMI en Java, CORBA de forma multilenguaje) que permiten invocar métodos de objetos ubicados en procesos remotos como si fueran locales.

3.6 Servicios web y APIs (SOAP, REST y gRPC)

Estilos de comunicación sobre los que se apoya el proyecto: SOAP, basado en XML y con contrato formal (WSDL); REST, orientado a recursos sobre HTTP; y gRPC, basado en HTTP/2 y Protocol Buffers.

3.7 Sincronización de archivos

Mecanismo por el cual los cambios en el árbol de directorios de un dispositivo cliente se propagan al repositorio central y a las demás copias, manteniendo una versión del recurso en el servidor.

3.8 Directorio activo

Servicio centralizado (LDAP / Active Directory) que administra los perfiles de usuario: autenticación, autorización, mapa de servicios y árboles de archivos.

3.9 Seguridad informática distribuida

Conjunto de mecanismos —cifrado, autenticación, integridad, disponibilidad y trazabilidad— que protegen la comunicación entre procesos y el acceso a los recursos.

3.10 Monitoreo de infraestructura

Recolección continua de métricas de uso (CPU, memoria, almacenamiento, carga de trabajos, tiempos de ejecución y disponibilidad) mediante exportadores por nodo, una base de series temporales y paneles de visualización y alertas.

4 Objetivos

4.1 Objetivo general

Desarrollar un sistema distribuido que proporcione servicios de computación de alto rendimiento (HPC) y gestión colaborativa de archivos, mediante una infraestructura en malla alojada en el Centro de Computación Avanzada de la Universidad Pontificia Bolivariana seccional Bucaramanga, para permitir a la microempresa UPB-CIENTÍFICA y a la comunidad científica ejecutar aplicaciones de procesamiento intensivo de manera segura y escalable, así como administrar, compartir y presentar recursos y datos científicos de forma centralizada.

4.2 Objetivos específicos

1. Documentar los requerimientos funcionales y no funcionales de la computación en malla, la computación de alto rendimiento y los paradigmas de comunicación distribuida pertinentes, con el fin de definir la arquitectura general del entorno de administración y ejecución de trabajos paralelos en MPI.
2. Desarrollar el entorno de administración y ejecución de código en paralelo con MPI sobre una infraestructura en malla (Grid), con un gestor del clúster en Java RMI que reciba el código fuente y los conjuntos de datos, planifique los trabajos y admita la integración de equipos internos y externos.
3. Desarrollar el servicio de sincronización de archivos en gRPC que cree en el servidor una copia síncrona de un directorio local, de forma programada en los equipos de escritorio y al detectar la red local en los dispositivos móviles, manteniendo la consistencia entre todas las copias.
4. Desarrollar el repositorio distribuido de archivos, fotos y videos y el servicio de compartición de archivos, con una interfaz de carga al Home del usuario, respetando el modelo de permisos Unix sobre un espacio parametrizado y con cuota máxima por usuario.
5. Desarrollar el servicio de álbum de fotografías, que funcione sobre la web y permita buscar, ordenar, presentar y compartir imágenes interna y externamente, y el servicio de streaming de video, con funciones de ordenamiento, reproducción fluida, control de acceso y gestión de permisos.
6. Integrar los componentes del sistema mediante las APIs especificadas para el proyecto (SOAP, JavaScript, Java RMI, gRPC y REST), garantizando la interoperabilidad entre los servicios desplegados en los distintos servidores.

7 Desarrollar un servicio de directorio activo para la administración centralizada de los perfiles de usuario, incluyendo autenticación, autorización, mapa de servicios y árboles de archivos.

8 Implementar mecanismos de seguridad informática mediante cifrado asimétrico GPG para las copias de seguridad, autenticación de los clientes basada en token y análisis de vulnerabilidades de la aplicación web en su ambiente de producción, con los niveles de riesgo asumidos documentados.

9 Implementar un módulo de monitoreo integral, accesible vía web, que reporte en tiempo real la ocupación de recursos, la carga de trabajos, los tiempos de ejecución y la disponibilidad de los servicios, con históricos y alertas configurables.

5 Metodología

El proyecto adopta un modelo de desarrollo iterativo e incremental (Sommerville, 2011), en el que el ciclo de vida del software —análisis, diseño, implementación, pruebas e integración— se recorre completo en cada iteración. Esta elección responde a un proyecto de infraestructura distribuida con múltiples servicios desarrollados en paralelo por un equipo de seis integrantes, que entrega en cada iteración un incremento funcional y verificable, alineado con las entregas de la asignatura (bitácoras en las semanas 8 y 16, producto en la semana 14). Antes de escribir código se fijaron los contratos de cada servicio (WSDL, .proto, interfaces RMI y REST), de modo que cada subequipo pudiera avanzar contra simulaciones de los servicios que consume.

El desarrollo se organiza en cuatro iteraciones: la Iteración 1 cubre la infraestructura Grid base, el directorio activo y la configuración del clúster HPC; la Iteración 2, los servicios File Sync (gRPC) y Shared File sobre el repositorio distribuido; la Iteración 3, los servicios Photo Album y Streaming; y la Iteración 4, el servicio MPI Jobs (gestor del clúster en Java RMI) y el módulo de monitoreo. Cada iteración termina con una revisión del registro de riesgos y con la integración incremental de los servicios construidos sobre la versión anterior del sistema.

6 Resultados

A la semana 14 del periodo 2026-20 el sistema se encuentra integrado y probado de extremo a extremo en un entorno local que reproduce el despliegue del Centro de Computación Avanzada (máquinas virtuales VirtualBox para el clúster y para el servicio de sincronización). La Tabla 1 resume el estado de cada componente frente a los requisitos del enunciado.

Componente (Figura 1)	**Tecnología**	**Estado**	**Evidencia**
Authentication / Users (directorio activo)	PHP 8 + SOAP + OpenLDAP	Implementado; despliegue en la VM del equipo pendiente	Token firmado con vigencia y revocación; mapa de servicios por perfil (validarAccesoServicio)
Web Server JavaScript (portal y SPA)	Node.js 20 + Nginx	Implementado	Pasarela única del navegador; CSP, límite de intentos de inicio de sesión
Repository (Distributed Storage)	Node.js + PostgreSQL	Implementado	Home por usuario con cuota parametrizada; permisos Unix
Shared File	Node.js (REST)	Implementado	Compartir archivos y directorios con usuarios y grupos; herencia de directorios

Componente (Figura 1)	Tecnología	Estado	Evidencia
Photo Album y Streaming	Node.js + FFmpeg	Implementado; compartir imágenes con personas externas pendiente	Orden en servidor, búsqueda, miniaturas, reproducción por rangos, contenido compartido; 20/20 pruebas
File Sync (gRPC Server Go Sync)	Go + gRPC	Implementado	Sincronización de tres vías con resolución de conflictos; programada (escritorio) y al detectar la Wi-Fi local (móvil); 19/19 pruebas
MPI Jobs (RMI Server Java Cluster)	Java 17 + RMI	Parcial	Gestor con cola por prioridad, token y mapa de servicios; pendiente la ejecución con mpirun en el clúster a partir de código fuente y datasets, y el registro de nodos externos de la malla
Monitoreo	Prometheus + Grafana	Implementado	CPU, memoria, almacenamiento, carga y tiempos de trabajos, disponibilidad; alertas e históricos; 32/32 pruebas
Seguridad	GnuPG, ZAP, nmap	Implementado; análisis en producción pendiente	Respaldos cifrados con llave pública; análisis de vulnerabilidades con niveles de riesgo asumidos
Clientes	Web, Python (Windows/Linux), Android	Implementado	Los tres clientes consumen los servicios reales

Los pendientes que condicionan el cumplimiento completo del enunciado son: la ejecución real de los trabajos MPI en el clúster a partir del código fuente y los conjuntos de datos recibidos, con el resultado guardado en el Home del usuario; la integración de equipos externos a la malla; la opción de compartir imágenes con personas externas; el despliegue del directorio activo en su máquina virtual; y la repetición del análisis de vulnerabilidades sobre el ambiente de producción.

7 Cronograma

La Tabla 2 presenta el cronograma del proyecto para las 17 semanas del periodo académico 2026-20, calculado a partir del inicio de clases el 14 de julio de 2026. En la versión final del artículo se actualizará con las fechas efectivamente ejecutadas, conforme lo exige el formato de entrega del curso.

Sem.	Fechas 2026	Fase	Hito principal
1	14 – 20 jul	Planificación	Roles del equipo, alcance técnico y tecnologías validadas.
2	21 – 27 jul	Planificación / Diseño	Arquitectura general y contratos de API entre servicios.
3	28 jul – 3 ago	Diseño	Modelo de directorio activo, permisos Unix y esquema de seguridad.
4	4 – 10 ago	Iteración 1	Infraestructura Grid base y directorio activo desplegados.
5	11 – 17 ago	Iteración 1	Clúster HPC configurado; primeras pruebas de ejecución MPI.
6	18 – 24 ago	Iteración 2	Servicio File Sync (gRPC) funcional.
7	25 – 31 ago	Iteración 2	Repositorio distribuido y servicio Shared File funcionales.

Sem.	Fechas 2026	Fase	Hito principal
8	1 – 7 sep	Cierre de iteración	Primer informe de seguimiento (bitácora).
9	8 – 14 sep	Iteración 3	Servicio Photo Album desplegado.
10	15 – 21 sep	Iteración 3	Servicio Streaming funcional.
11	22 – 28 sep	Iteración 4	Servicio MPI Jobs (Java RMI) integrado con el clúster.
12	29 sep – 5 oct	Iteración 4	Módulo de monitoreo desplegado.
13	6 – 12 oct	Integración	Integración completa y pruebas end-to-end.
14	13 – 19 oct	Producto	Entrega del producto funcional completo.
15	20 – 26 oct	Participación institucional	Evento obligatorio (20-22 oct.) y hardening de seguridad.
16	27 oct – 2 nov	Cierre	Segundo informe, artículo científico final y presentación JSPI.
17	3 – 9 nov	Sustentación	Sustentación de carácter individual.

8 Discusión y conclusiones

El principal reto del proyecto ha sido la integración oportuna de servicios desarrollados por subequipos distintos en tecnologías diferentes. Fijar los contratos antes de implementar y validar cada servicio contra un validador de tokens intercambiable (simulado o SOAP) permitió avanzar en paralelo e integrar sin rehacer interfaces. La decisión de que todos los servicios que guardan datos —incluida la sincronización— pasen por la API del repositorio, en vez de escribir directamente en el sistema de archivos, mantuvo un único punto que aplica cuotas y permisos Unix.

El trabajo pendiente se concentra en el núcleo HPC: completar la ejecución paralela real sobre el clúster a partir del código fuente y los datos recibidos, y la integración de equipos externos a la malla. Como contribución, este trabajo documenta un caso aplicado de integración entre múltiples paradigmas de sistemas distribuidos —objetos distribuidos, servicios web, RPC binario, paso de mensajes y computación en malla— dentro de una única solución coherente, que puede servir de referencia para futuros desarrollos de infraestructura científica en la UPB seccional Bucaramanga. Las conclusiones definitivas se completarán con las métricas del módulo de monitoreo tras la entrega del producto.

Agradecimientos. Los autores agradecen al docente MSc. Lenin Javier Serrano Gil por la orientación brindada durante el desarrollo de este proyecto, así como a la Facultad de Ingeniería de Sistemas e Informática de la UPB seccional Bucaramanga por el acceso al Centro de Computación Avanzada.

Conflicto de intereses. Los autores declaran no tener conflictos de interés relacionados con el contenido de este artículo.

Referencias bibliográficas

1 Adepoju, S., & Ologunagba, G. F. (2025). A systematic review and comparative evolution of grid computing and modern cloud computing infrastructures. International Journal of Scientific Research in Computer Science and Engineering, 13(5), 43–51.

2 AlHafez, N., & Kurdi, A. (2025). Parallel paradigms in modern HPC: A comparative analysis of MPI, OpenMP, and CUDA. arXiv preprint arXiv:2506.15454.

3 Babu, R., Abraham, G., & Borasia, K. (2013). A review on securing distributed systems using symmetric key cryptography. arXiv preprint arXiv:1303.0351.

4 Capra, R. (2014). File synchronization and sharing: User practices and challenges. Proceedings of the American Society for Information Science and Technology, 51(1), 1–10.

5 Castillo, P. A., Bernier, J. L., Arenas, M. G., Merelo, J. J., & García-Sánchez, P. (2011). SOAP vs REST: Comparing a master-slave GA implementation. arXiv preprint arXiv:1105.4978.

6 Guerraoui, R., & Schiper, A. (1996). Fault-tolerance by replication in distributed systems. En A. Strohmeier (Ed.), Reliable Software Technologies — Ada-Europe '96 (LNCS, vol. 1088, pp. 38–57). Springer.

7 Juric, M. B. (2000). Comparison of CORBA and Java RMI based on performance analysis. Actas de los Workshops de ECOOP.

8 Kunz, P. (2022). HPC job-monitoring with SLURM, Prometheus and Grafana (Tesis de pregrado). Universidad de Basilea.

9 Mehdi, Z., & Ragab-Hassen, H. (2016). File synchronization systems survey. Actas de ICCSEA, SPPR, UBIC 2016, 29–38.

10 Munoz, C., & Zalewski, J. (2001). Architecture and performance of Java-based distributed object models: CORBA vs RMI. Real-Time Systems, 21(1).

11 Natali, L., & Merani, M. L. (2015). Adaptive streaming in P2P live video systems: A distributed rate control approach. arXiv preprint arXiv:1510.04472.

12 Priyadarshini, S. B. B. (2022). Security in distributed system: A review perspective.

13 Sommerville, I. (2011). Software Engineering (9.ª ed.). Addison-Wesley.

14 Sungkar, A., & Kogoya, T. (2020). A review of grid computing. Computer Science & IT Research Journal, 1(1), 1–6.

15 UPB. (2020). Carta descriptiva del curso: Sistemas distribuidos. Código DA-FO-085N, versión 3_3.

16 Waheed, M. H. B., Jamil, F., Qayyum, A., Jamil, H., Cheikhrouhou, O., Ibrahim, M., Bhushan, B., & Hmam, H. (2021). A new efficient architecture for adaptive bit-rate video streaming. Sustainability, 13(8), 4541.

17 Woo, T. Y. C., & Lam, S. S. (1992). Authentication for distributed systems. IEEE Computer, 25(1), 39–51.